МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования
«Московский авиационный институт (национальный исследовательский университет)»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
№ 2

Дисциплина:

Операционные системы и архитектура компьютера

Тема:

Управление процессами и обмен данными через каналы
Вариант 4

Выполнил:

студент группы М8О-210БВ-25
Македонский Александр Сергеевич

Москва, 2026 г.

1. Цель работы

Освоить создание и завершение процессов, обмен данными через неименованные каналы и обработку системных ошибок. Разработать на языке C программу из двух отдельных исполняемых файлов: родитель передаёт команды, а потомок выполняет вычисления и сохраняет результаты.

Вариант 4 группы 1: первая строка ввода задаёт имя выходного файла. Каждая следующая строка содержит произвольное количество чисел float. Потомок последовательно делит первое число на остальные: для «100 2 5» результат равен (100 / 2) / 5 = 10. При нулевом делителе проверку выполняет потомок, после чего оба процесса завершаются.

2. Краткое описание архитектуры решения

Решение написано на C11 с использованием POSIX API, целевая ОС — Linux. parent.c создаёт два канала, вызывает fork и запускает child через execvp. child.c открывает файл и записывает результаты через dprintf. Разбор чисел вынесен в division.c; чтение строк, полная запись, сигналы и ожидание потомка — в common.c. Сборка задана в src/Makefile.

2.1 Схема взаимодействия компонентов

```
Пользователь → parent
parent → pipe1 → stdin программы child
child → выходной файл с результатами
stdout программы child → pipe2 → parent
parent и child → stderr с PID
```

Рисунок 1 — Поток данных и сообщений между процессами

Имя файла передаётся потомку аргументом execvp. Через dup2 вход потомка связывается с pipe1, а выход — с pipe2. Родитель оставляет запись в pipe1 и чтение pipe2; потомок — чтение pipe1 и запись pipe2. Ненужные концы каналов закрываются. Общая память и дополнительные потоки не используются.

Потомок отправляет однобайтовые ответы: R — файл открыт и можно начинать; A — команда обработана; I — строка отвергнута; Z — нулевой делитель; F — ошибка. После каждой команды родитель ждёт ответ, поэтому следующая команда не передаётся после Z. По окончании ввода родитель закрывает pipe1 и вызывает waitpid для созданного потомка.

3. Листинг ключевых фрагментов кода

Ниже приведены фрагменты текущих исходников. Полный код находится в каталоге src; адрес репозитория:

https://github.com/amakedoni/oci

3.1 Создание каналов и запуск потомка

В parent.c, строки 64–85, оба канала создаются до fork. В дочерней ветви dup2 связывает стандартные потоки с каналами, закрываются исходные дескрипторы и запускается отдельная программа child. Ошибка execvp приводит к выходу с кодом 127.

```
    if (pipe(commands) == -1) {
        perror("parent: pipe1");
        goto cleanup;
    }
    if (pipe(replies) == -1) {
        perror("parent: pipe2");
        goto cleanup;
    }
    child = fork();
    if (child == -1) {
        perror("parent: fork");
        goto cleanup;
    }
    if (child == 0) {
        if (dup2(commands[0], STDIN_FILENO) == -1 ||
            dup2(replies[1], STDOUT_FILENO) == -1)
{ perror("child: dup2"); _exit(1); }
        if (close_pipes(commands, replies) == -1) _exit(1);
        char *const arguments[] = {program, filename, NULL};
        execvp(program, arguments);
        perror("child: execvp");
        _exit(127);
    }
```

Листинг 1 — Каналы, перенаправление потоков и execvp

Успешный execvp заменяет программу в уже созданном процессе. Он не создаёт ещё один процесс. Родитель после fork продолжает собственную ветвь и проверяет ответ R перед отправкой первой команды.

3.2 Разбор чисел и последовательное деление

В division.c, строки 8–33, strtof возвращает очередное число float, а через указатель end сообщает адрес конца числа. Проверяются границы элемента, диапазон и конечность значения. Первое число становится результатом, остальные используются как делители.

```
enum calculation divide_line(const char *line, float *result)
{
    const char *position = line;
    int first = 1;
    while (*position != '\0') {
        while (isspace((unsigned char)*position)) ++position;
        if (*position == '\0') break;
        char *end;
        errno = 0;
        float value = strtof(position, &end);
        if (end == position || (*end != '\0' && !isspace((unsigned
char)*end)))
            return CALC_INVALID;
        if (errno == ERANGE || !isfinite(value)) return
CALC_RANGE;
        if (first) {
            *result = value;
            first = 0;
        }
        else {
            if (value == 0.0f) return CALC_ZERO;
            *result /= value;
            if (!isfinite(*result)) return CALC_RANGE;
        }
        position = end;
    }
    return first ? CALC_EMPTY : CALC_OK;
}
```

Листинг 2 — Вычисление результата одной команды

Сравнение с 0.0f обнаруживает и положительный, и отрицательный ноль. Ноль первым операндом допустим: «0 5» даёт 0. Один операнд записывается без изменения, пустая строка пропускается. Вывод использует FLT_DECIMAL_DIG; вычисления float подвержены округлению, а очень малый промежуточный результат может округлиться до нуля.

3.3 Завершение при делении на ноль

В child.c, строки 60–65, потомок сообщает о нулевом делителе через pipe2 и завершает цикл с кодом 2. Этот ответ формируется до записи результата ошибочной команды.

```
        if (calculation == CALC_ZERO) {
            if (dprintf(STDERR_FILENO, "[child pid=%ld] line=%zu:
division by zero; stopping\n", (long)getpid(), number) < 0) goto
cleanup;
            if (send_reply(ZERO) == -1) goto cleanup;
            result = EXIT_ZERO_DIVISION;
            break;
        }
```

Листинг 3 — Уведомление родителя о нулевом делителе

В parent.c, строки 122–126, ответ ZERO прекращает отправку команд. Общий участок cleanup закрывает каналы, ожидает потомка и освобождает память.

3.4 Динамическая память и обработка сигналов

read_line начинает с 256 байт и удваивает вместимость при нехватке места. В common.c 100–110 временный указатель grown позволяет проверить realloc, сохранив прежний блок при отказе.

```
        if (*size + 1 == capacity) {
            if (capacity > SIZE_MAX / 2) {
                errno = ENOMEM;
                goto fail;
            }
            size_t next = capacity * 2;
            char *grown = realloc(*line, next);
            if (grown == NULL) goto fail;
            *line = grown;
            capacity = next;
        }
```

Листинг 4 — Безопасное увеличение буфера строки

Побайтовое чтение упрощает код, увеличивая число read. Строка освобождается после обработки. write_all дописывает частичную запись. poll позволяет parent заметить исчезновение child во время ожидания ввода.

SIGINT и SIGTERM сохраняются во флаге interrupted; уборка выполняется вне обработчика. SIGPIPE игнорируется, чтобы программа могла обработать EPIPE при записи без читателя.

Проверяются результаты создания каналов и процесса, перенаправления потоков, открытия и записи файла, выделения памяти и ожидания потомка. waitpid повторяется при EINTR. При обработанном прерывании код родителя равен 128 + номер сигнала; например, для SIGTERM — 143.

4. Результаты работы

Приведены сохранённые результаты реальных запусков из report/logs. Текущая локальная среда: macOS 27.2 arm64, Apple Clang 21.0.0, Python 3.9.6. Код использует POSIX API для Linux; фактическая трассировка Linux strace здесь не выполнялась.

4.1 Консольный вывод и логи

Обычный запуск выполняется из корня репозитория. Первая строка INPUT содержит имя файла, остальные — команды. STDERR содержит диагностику, OUTPUT FILE — полученные данные. Существующий выходной файл при запуске очищается.

```
$ ./src/build/parent < src/examples/normal.in
INPUT:
src/build/results.txt
100 2 5
7.5 2.5
-24 2 -3
0 5
0.5
STDERR:
[parent pid=27732] child=27733; pipe1 sends commands, pipe2
receives status
[child pid=27733] ppid=27732; output=src/build/results.txt; ready
[child pid=27733] line=1: result=10
[child pid=27733] line=2: result=3
[child pid=27733] line=3: result=4
[child pid=27733] line=4: result=0
[child pid=27733] line=5: result=0.5
[parent pid=27732] child=27733 exited status=0; reaped
Exit code: 0
OUTPUT FILE:
10
3
4
0
0.5
```

PID 27732 относится к родителю, PID 27733 — к потомку; ppid потомка совпадает с PID родителя. Результаты 10, 3, 4, 0 и 0.5 соответствуют входным строкам. Сообщение exited status=0; reaped подтверждает получение статуса потомка через waitpid.

Во втором запуске после успешной команды «12 2» поступает строка «8 0». Потомок обнаруживает нулевой делитель и сообщает об остановке; следующая команда «99 3» не обрабатывается.

```
$ ./src/build/parent < src/examples/zero.in
INPUT:
src/build/zero-result.txt
12 2
8 0
99 3
STDERR:
[parent pid=27734] child=27735; pipe1 sends commands, pipe2
receives status
[child pid=27735] ppid=27734; output=src/build/zero-result.txt;
ready
[child pid=27735] line=1: result=6
[child pid=27735] line=2: division by zero; stopping
[parent pid=27734] division by zero reported by child; stopping
[parent pid=27734] child=27735 exited status=2; reaped
Exit code: 2
OUTPUT FILE:
6
```

Оба процесса завершаются с кодом 2, в файле остаётся только 6. Разные PID и сообщение reaped показывают отдельные процессы и ожидание потомка родителем. Полные журналы сохранены в report/logs/normal.log и report/logs/zero.log.

В запуске src/examples/invalid.in первые четыре строки отклоняются: две содержат неверные токены, ещё две — NaN и число вне диапазона float. После этого команда «9 3» даёт 3, обмен завершается с кодом 0.

```
[parent pid=27736] child=27737; pipe1 sends commands, pipe2
receives status
[child pid=27737] ppid=27736; output=src/build/invalid-result.txt;
ready
[child pid=27737] line=1: invalid float token; skipped
[child pid=27737] line=2: invalid float token; skipped
[child pid=27737] line=3: float out of range; skipped
[child pid=27737] line=4: float out of range; skipped
[child pid=27737] line=5: result=3
[parent pid=27736] child=27737 exited status=0; reaped
Exit code: 0
OUTPUT FILE:
3
```

4.2 Проверки краевых случаев

В каждом из двух сохранённых запусков 15 тестов успешны, один пропущен: устройство /dev/full отсутствует на macOS. AddressSanitizer и UndefinedBehaviorSanitizer не сообщили об ошибках в выполненных сценариях. Ниже приведены итоговые строки журналов.

```
$ make -C src test
Ran 16 tests in 0.808s
OK (skipped=1)
$ make -C src sanitize
Ran 16 tests in 1.212s
OK (skipped=1)
```

Проверены дробные и отрицательные числа, ноль первым операндом, делители ±0, неверные токены, NaN, бесконечность, встроенный NUL и выход за диапазон. Строка из числа 42 и 100 000 единиц даёт результат 42; результаты 500 команд совпадают с ожидаемыми с относительным допуском 3e-7. Последняя строка без перевода строки обрабатывается.

Аварийные сценарии включают пустое имя файла, ошибку открытия, отсутствие программы child со статусом 127, лимит дескрипторов 5, обрыв связи и исчезновение потомка при ожидании ввода. SIGTERM завершает родителя с кодом 143 и ожиданием потомка. При нулевом делителе остановка происходит даже при ещё открытом пользовательском вводе.

Сборка с -Wall -Wextra -Wpedantic -Werror и статический анализ clang --analyze четырёх модулей C прошли без замечаний. Полные журналы: report/logs/build.log, tests.log и sanitizers.log.

4.3 Демонстрация системных вызовов в Linux

strace — внешняя утилита Linux, а не функция внутри программы. Она показывает системные вызовы и сигналы. На Linux после сборки при установленном strace из корня репозитория выполняется команда ниже. Это инструкция для демонстрации; фактическая Linux-трасса локально не получена.

```
strace -f -o src/build/trace.log ./src/build/parent <
src/examples/zero.in
```

Ключ -f включает потомка, -o сохраняет трассу. Создание каналов соответствует parent.c 64–71; fork — строке 72; dup2 — 78–79; execvp, отображаемый через execve, — 82. Открытие файла находится в child.c 32, запись результата через dprintf — 74. При нуле child.c 62 вызывает send_reply: write в child.c 14 через common.c 33 отправляет Z в stdout, то есть pipe2. Ожидание waitpid находится в common.c 126 и на Linux обычно видно как wait4. При этом примере код завершения равен 2, а в файле остаётся 6.

5. Выводы

Реализованы отдельные программы родителя и потомка с двумя каналами, вычислением последовательного деления float и записью результатов в файл. Нулевой делитель проверяется потомком и останавливает оба процесса. Логи с PID, тесты и санитайзеры подтвердили работу на macOS. Освоены создание процессов, перенаправление потоков, обмен через каналы, освобождение памяти и ожидание потомка.